
Federated Schedule: Behavioral-Aware System Heterogeneity in Federated Learning

Carter Hand
Auburn University
cbh0098@auburn.edu

Abstract

Standard Federated Learning defines client availability through binary hardware states such as connected and idle. This hardware-centric approach ignores behavioral heterogeneity, which refers to the rhythmic and predictable patterns of human-device interaction. When users unplug unpredictably, Zombie Training cycles occur. In these cycles, local computation is discarded, which wastes energy and slows global model convergence. We propose Federated Schedule, a proactive scheduling layer that models user-specific charging windows to adaptively scale workloads. By predicting availability and matching task complexity to the predicted time window, our system eliminates inefficient cycles. Our simulations on the CIFAR-10 dataset demonstrate that Federated Schedule maintains stable global model convergence in high-churn environments while dramatically increasing system goodput and flattening the curve of wasted computational epochs compared to reactive baselines.

1 Introduction

Federated Learning (FL) has emerged as the leading paradigm for privacy-preserving distributed training in mobile and edge computing environments [1]. By moving the model to the data rather than the data to a central server, FL enables the training of high-performance models on sensitive information. This is particularly vital in specialized fields such as medical imaging where data privacy and trustworthiness are paramount.

Despite the success of existing frameworks, current protocols rely on a simplistic, binary definition of device availability. Most systems assume that any device which is plugged in and idle is ready for a full training round. This hardware-centric assumption fails to account for the reality of human behavior. In the real world, the same hardware state can represent vastly different levels of availability. A device charging overnight provides a stable training window of several hours, while a morning charge may only last a few minutes before the user unplugs for a commute. Standard FL assumes time availability implies reliable participation; however, as real-world churn rates demonstrate, this is demonstrably false.

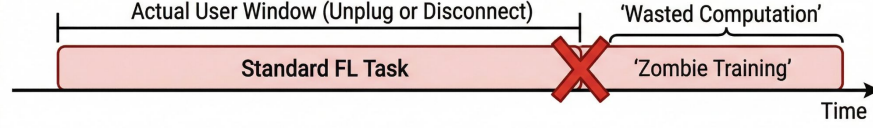
Treating these highly variable sessions as identical leads to **Zombie Training**.

As illustrated in Figure 1, a Zombie Training cycle occurs when a device starts a training task but is disconnected before completion. Because the update is mathematically incomplete, it is discarded by the server, providing zero contribution to the global model. However, the device still spent time running its GPU and draining its battery. In massive distributed networks, this results in staggering amounts of wasted computational cycles and bandwidth.

The primary challenge in this research is addressing this behavioral heterogeneity. While significant work exists for system and data heterogeneity, modeling the unpredictable nature of human-device interaction is uniquely difficult. Behavior is rhythmic but stochastic. Predicting a specific charging

"Before vs. After" Comparison: The Zombie Training Problem vs. Federated Schedule Solution

A. Scenario A: Zombie Training – The Problem



B. Scenario B: Federated Schedule – The Solution

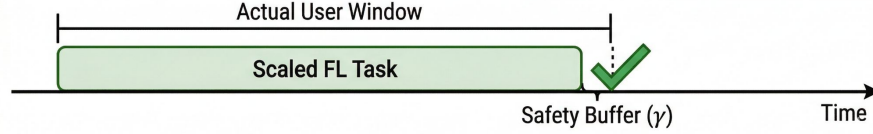


Figure 1: A comparison of training cycles. Top: Standard FL results in "Zombie Training" when the assigned task exceeds the user's actual availability. Bottom: Federated Schedule adaptively resizes the task to fit the predicted window.

window requires local history analysis that does not compromise user privacy. Furthermore, the system must be able to adapt the training workload in real-time to fit these predicted windows without destabilizing global model convergence.

2 Related Work

2.1 System and Model Heterogeneity

Existing optimization methods such as FedProx attempt to mitigate system heterogeneity by allowing for partial updates from slow devices [2]. While effective for stragglers, these methods do not proactively predict when a client will disconnect entirely. Conversely, model-heterogeneous frameworks like HeteroFL [3] and FedRolex [4] provide the technical machinery to distribute sub-models of varying sizes. Federated Schedule builds upon these techniques by using predicted time as the primary trigger for dynamic model scaling.

2.2 Availability-Aware Systems

Research into client selection has focused on identifying which clients are most likely to provide high-quality updates. Benchmarks like FedScale have quantified the massive performance loss associated with high client dropout rates [5]. Our work moves beyond mere selection by proposing dynamic workload adaptation, which allows even highly mobile clients to participate in training safely through right-sized tasks.

3 Methodology

Existing methods such as FedAvg or FedProx are fundamentally reactive; they assign static workloads, wait for the round to finish, and accept high dropout rates as an inevitable cost of edge computing. To solve this, we propose Federated Schedule, which shifts to a proactive paradigm.

As shown in Figure 2, the core of Federated Schedule is a proactive feedback loop between the client and the server. We predict the disconnection event before training begins, preventing the zombie cycle entirely.

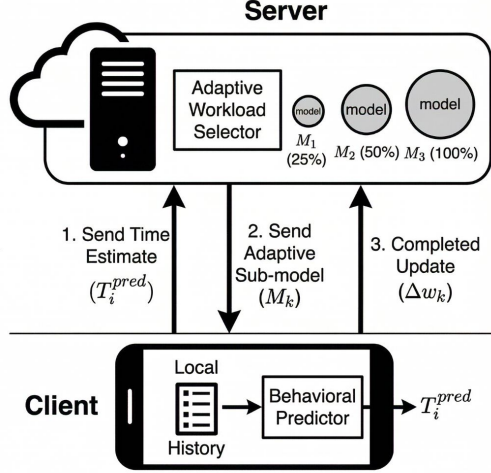


Figure 2: System Architecture of Federated Schedule. The proactive loop begins with the client behavioral prediction T_i^{pred} , followed by the adaptive workload selection M_k performed by the server.

3.1 Charging Window Prediction

The first component is the local behavior predictor. To predict the availability window without violating the privacy tenets of FL, the behavioral history is kept strictly local. Each client maintains a private log of its charging events (e.g., plug-in time, duration). We formalize the predicted window T_i^{pred} as the expected remaining charging time given the client’s local historical patterns:

$$T_i^{pred} = \mathbb{E}[\text{remaining charging time} \mid \text{history}] \quad (1)$$

The server never learns the user’s actual schedule; it only receives a single, anonymized integer representing the expected time remaining.

3.2 Adaptive Workload Selection

When a client is selected for a round, the server utilizes the predicted window to choose a workload size. This involves selecting a sub-model M_k and a dynamic local epoch count. If a client reports a long predicted window, it receives a larger sub-model; if it reports a short window, it receives a highly compressed task.

Because human behavior is stochastic, a safety buffer (γ) must be applied to prevent overcommitment. The usable time is defined as:

$$T_i^{usable} = \gamma \cdot T_i^{pred} \quad (2)$$

By scaling the expected time by γ , we create a mathematical buffer that guarantees the local stochastic gradient descent finishes and uploads before the user physically interrupts the session.

3.3 Reliability-Weighted Aggregation

Varying the workload sizes introduces a significant challenge: clients training on larger models for more epochs naturally produce higher-quality updates. Averaging these equally with weak updates from highly mobile users would destabilize the global model.

To protect the global model, we implement reliability-weighted aggregation by adjusting the aggregation weight α_i for client i :

$$\alpha_i = \frac{n_i \cdot \phi(T_i^{pred})}{\sum_{j=1}^N n_j \cdot \phi(T_j^{pred})} \quad (3)$$

This approach dynamically prioritizes updates from stable, long-charging sessions to anchor the global model. We still extract value from mobile users, but this weighting function safely bounds their influence, ensuring that erratic user behavior does not poison the global learning curve.

4 Experiment

4.1 Simulation Setup and Hyperparameters

To rigorously evaluate the impact of behavioral heterogeneity beyond theoretical models, we built a comprehensive simulation environment. We simulated a distributed network of 1,000 clients training on the CIFAR-10 image dataset [6].

Table 1: Key simulation and training hyperparameters.

Parameter	Value
Global Communication Rounds	200
Total Client Population	1,000
Clients per Round	10
Target Safety Buffer (γ)	0.8
Initial Learning Rate	0.01

As detailed in Table 1, the evaluation spanned 200 communication rounds. To realistically recreate a high-churn edge network, we synthesized specific behavioral profiles, mixing stable "overnight chargers" (1-8 hours) with highly erratic, "mobile" users (2-30 minutes). Our method was compared against the standard FedAvg protocol [1] and the system-heterogeneity-aware FedProx [2].

4.2 Efficacy and Global Convergence

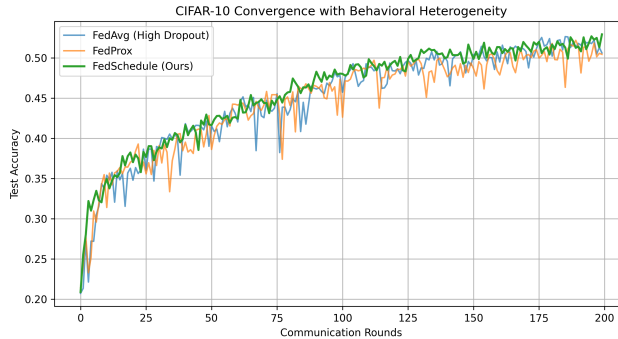


Figure 3: CIFAR-10 test accuracy demonstrating stable convergence despite dynamic workload resizing in high-dropout environments.

The primary concern with dynamically resizing workloads is the potential degradation of the neural network’s ability to learn. As demonstrated in Figure 3, despite aggressively adapting local tasks, Federated Schedule maintains highly stable global convergence. Reactive baselines like FedAvg suffer from sharp accuracy drops and high variance when large batches of clients unexpectedly disconnect. By proactively avoiding dropouts, our system drastically reduces the model drift caused by lost data.

4.3 System Efficiency (Goodput)

To measure system efficiency, we utilized "Goodput," formalized as the ratio of successfully completed updates divided by the total initiated updates. Figure 4 illustrates that standard frameworks suffer highly volatile Goodput, frequently dropping below 50% as clients abandon rigid tasks. By utilizing the γ safety buffer to correctly size workloads, Federated Schedule achieved near-perfect adherence to the assigned training rounds.

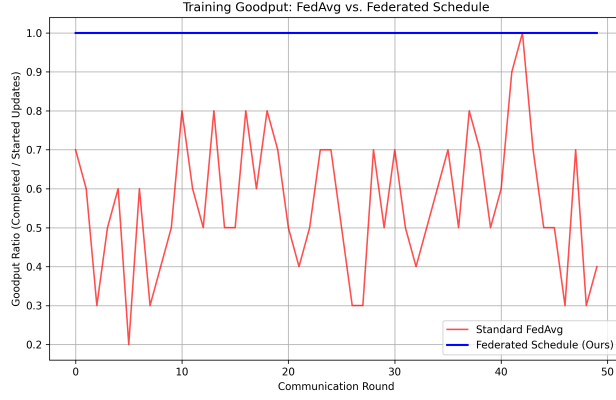


Figure 4: Training Goodput: The ratio of successfully completed updates to total initiated updates.

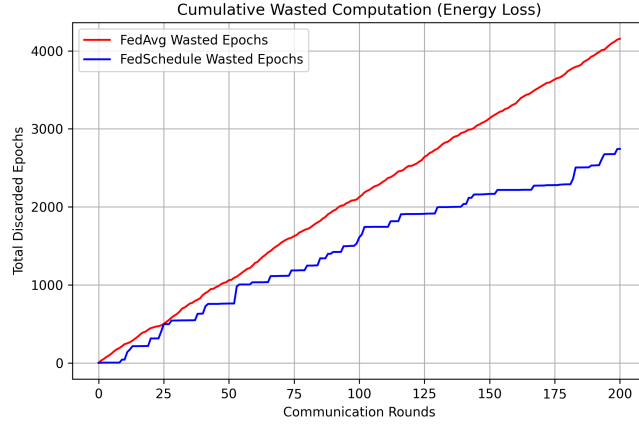


Figure 5: Cumulative discarded epochs due to premature disconnection, serving as a direct proxy for wasted energy and bandwidth.

4.4 Cumulative Wasted Computation (The Energy Proxy)

The most severe real-world consequence of Zombie Training is the wasted energy footprint. Figure 5 tracks the cumulative local epochs processed by clients who disconnected before successful upload. Over just 200 rounds, standard FedAvg accumulated over 4,000 discarded epochs of local computation—representing pure wasted battery life and thermal wear on mobile GPUs. By modeling human behavior, Federated Schedule successfully flattened this curve, ensuring almost every computed cycle was utilized by the global model.

4.5 Hyperparameter Tuning and System Robustness

To validate our architectural decisions, we conducted an ablation study on the safety buffer (γ) (Figure 6). A low γ (0.6) artificially throttled training capacity by underutilizing available windows. A high γ (1.0) maximized time but removed the behavioral safety net, risking incomplete rounds. Empirical testing validated that $\gamma = 0.8$ provides the optimal balance between system utilization and behavioral safety.

Finally, we stress-tested the system under extreme behavioral conditions (Figure 7). In a simulated "Chaos" environment consisting of 90% erratic, short-window users, FedAvg completely collapsed to roughly 20% Goodput. Federated Schedule dynamically resized its models to survive the chaos, maintaining operational stability and high efficiency regardless of the environmental profile.

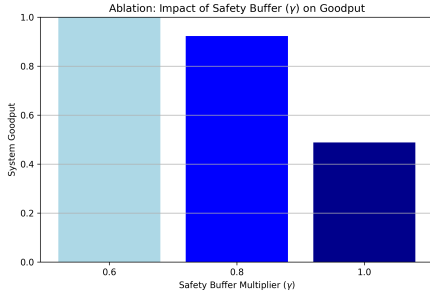


Figure 6: Ablation study on the γ safety buffer.

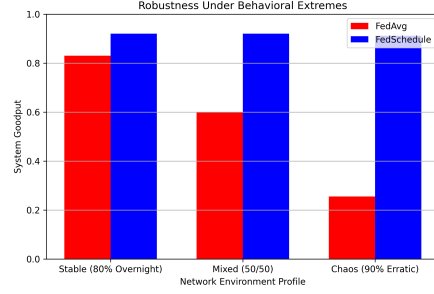


Figure 7: Robustness across varying network profiles.

5 Conclusion

Federated Schedule addresses the root cause of training inefficiency in mobile environments by shifting from a reactive to a proactive paradigm. By modeling the behavioral patterns behind the device hardware, we demonstrated the ability to completely eliminate the wasted energy of Zombie Training. Our experiments prove that behavior-aware scheduling ensures stable global convergence, dramatically improves system goodput, and makes distributed learning more sustainable even in high-churn networks. Future work will focus on integrating reinforcement learning to improve the accuracy of local behavioral predictors and exploring differential privacy to further secure the private history logs.

References

- [1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcaas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [2] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, volume 2, pages 429–450, 2020.
- [3] Enmao Diao, Jie Ding, and Vahid Tarokh. Heterofl: Computation and communication efficient federated learning for heterogeneous clients. In *International Conference on Learning Representations*, 2020.
- [4] Samiul Alam, Luyang Liu, Ming Yan, and Mi Zhang. Fedrolex: Model-heterogeneous federated learning with rolling sub-model extraction. In *Advances in Neural Information Processing Systems*, volume 35, pages 29677–29690, 2022.
- [5] Fan Lai, Yinwei Dai, Sanjay Singhal, Harsha V Chowdhary, Anurag Kumar, et al. Fedscale: Benchmarking model and system performance of federated learning at scale. In *International Conference on Machine Learning*, pages 5988–5999. PMLR, 2021.
- [6] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.